王启智 AI 应用作品集学习文档

版本：2026.10

这份文档不是让你背代码，而是帮助你在面试时回答：

1. 这个项目解决什么问题？
2. 每个模块为什么这样设计？
3. 出错时怎么定位？
4. 如何从 Demo 继续做成业务系统？

一、先建立整体认知

三个项目对应三个真实业务问题：

项目	业务问题	你要展示的能力
RAG 知识库	企业资料很多，员工找不到答案	文档处理、检索、回答边界
Text-to-SQL	业务人员不会写 SQL	Schema 理解、SQL 生成、安全执行
CodePilot Lite	开发者希望用自然语言完成工程任务	工具调用、权限控制、测试闭环

面试时不要只说“我用了某某框架”。要按照这个顺序讲：

```
业务问题 -> 输入 -> 处理流程 -> 输出 -> 风险 -> 验证方式
```

二、RAG 知识库

2.1 完整流程

```
文档
-> 解析
-> 切分 Chunk
-> 转成特征
-> 建立索引

用户问题
-> 转成特征
-> 混合检索
-> Top-K 结果
-> 交给模型
-> 带来源回答
```

2.2 代码阅读顺序

1. 先看 split_document()：理解为什么不能把整篇文档直接塞给模型。

2. 再看 tokenize() 和 hashed_embedding()：理解文本如何变成可比较的数值表示。
3. 再看 HybridRetriever.search()：理解词项匹配和向量相似度如何合并。
4. 最后看 MockLLM.answer()：理解为什么要把来源一起返回。

2.3 你必须会讲

为什么需要 Chunk？

长文档包含大量无关内容，直接检索会降低准确性；切成较小片段后，可以把更相关的内容送给模型。但 Chunk 过小会丢失上下文，所以需要根据文档结构、标题和段落长度调整。

为什么需要 Rerank？

向量检索通常先快速召回候选片段，但候选顺序不一定完全符合问题意图。Rerank 可以对候选结果再次排序，把最相关的内容放到上下文前面。

如何减少幻觉？

- 只允许模型基于检索内容回答
- 检索不到内容时拒答
- 返回来源文件和片段
- 对关键问题增加人工或规则校验

2.4 这个 Demo 的限制

本项目默认使用确定性哈希向量，不等同于生产环境的 Embedding 模型。这样做的目的是让项目无需下载模型也能运行。生产环境可以替换为真实 Embedding 服务，并增加离线评测集、召回率和答案正确性评测。

三、Text-to-SQL

3.1 完整流程

```
自然语言问题
-> 获取表结构和字段说明
-> 生成 SQL
-> SQL 安全校验
-> 数据库执行
-> 返回表格结果
-> 生成自然语言解释
```

3.2 代码阅读顺序

1. DemoDatabase._seed()：了解示例表和字段。
2. DemoDatabase.schema()：理解模型为什么需要 Schema。

3. RuleBasedSQLGenerator.generate()：理解自然语言到 SQL 的映射。
4. SafeSQLValidator.validate()：理解为什么生成 SQL 不能直接执行。
5. TextToSQLService.ask()：串起完整闭环。

3.3 必须会讲的安全问题

模型可能生成：

- 不存在的字段
- 错误的表名
- 业务含义错误的 SQL
- DELETE、UPDATE、DROP 等危险语句

所以本项目采取：

- 只允许 SELECT
- 禁止多语句执行
- 自动增加 LIMIT
- 使用独立的示例数据库
- 执行前做关键词和语法层面的校验

生产环境还要增加数据库账号权限隔离、SQL AST 解析、表级权限、字段级权限和查询超时。

四、CodePilot Lite

4.1 它不只是聊天机器人

普通聊天机器人只返回文本；代码代理需要使用工具：

```
模型
-> 选择工具
-> 工具执行
-> 返回结果
-> 模型继续判断
```

本项目的工具包括：

- list_files()：列出工作区文件
- read_file()：读取代码
- search_text()：搜索文本
- run_safe_command()：执行白名单命令
- apply_replace()：只生成 Patch 预览，不直接覆盖

4.2 为什么权限控制重要

如果 Agent 可以任意访问文件和执行命令，模型错误就可能变成数据删除或系统破坏。因此要做：

- 工作区路径限制
- 命令白名单
- 禁止 shell 拼接
- 危险命令拦截
- 修改文件前先展示 Patch
- 重要操作需要用户确认

4.3 这个 Demo 的下一步

当前版本使用规则 Agent，目的是先把工具和边界学清楚。下一步可以增加模型规划器，让模型输出结构化动作：

```
{
"tool": "read_file",
"arguments": {"path": "sample_project/calculator.py"}
}
```

收到动作后，程序仍然必须经过 Policy 检查，不能因为模型输出了 JSON 就直接执行。

五、如何学习代码

不要一次性通读所有文件。按以下顺序：

第一天：能运行

```
cd rag-knowledge-base
python3 app.py --question "员工申请年假需要提前多久？"

cd ../text2sql-assistant
python3 app.py --question "查询每个部门有多少员工"

cd ../codepilot-lite
python3 devagent.py "列出项目文件"
```

第二天：能改一个小功能

- RAG：修改 top_k，观察返回来源数量变化
- Text-to-SQL：新增一个自然语言规则
- CodePilot：增加一个只读工具

第三天：能解释限制

每个项目都回答：

1. 当前版本哪里不够好？
2. 如何测试？
3. 如何接入真实模型？
4. 如何部署？
5. 如何保护数据？

六、面试讲解模板

60秒项目介绍

> 我做了一个面向企业场景的 AI 应用原型，核心目标是把自然语言能力接入真实业务流程。系统包含输入处理、模型调用、工具或数据库执行、结果验证和错误反馈几个环节。我重点关注了两个问题：一是模型回答不能脱离上下文，二是模型生成的动作不能直接执行。因此在 RAG 项目中加入来源和拒答，在 Text-to-SQL
> 中加入只读校验，在代码代理中加入工作区和命令权限控制。

被问“你是不是只会调用 API”时

> 模型调用只是其中一层。我还需要处理数据结构、工具协议、权限、异常、测试和结果展示。这个作品集的重点就是把模型输出接到可控的业务动作上，并对失败情况做边界处理。

被问“你代码能力怎么样”时

> 我目前不是算法训练方向，重点是 Python AI 应用工程。我能独立阅读和修改接口、数据处理、检索、SQL 校验和工具调用代码，也在补充测试、日志和部署能力。遇到不熟悉的模块，我会先跑通、写最小测试，再逐步扩展。

七、简历可用描述

完成并理解代码后，可以根据真实情况修改为：

> AI应用作品集：从0-1实现企业知识库 RAG、Text-to-SQL
> 和终端式代码代理三个可运行原型，覆盖文档检索、SQL安全校验、工具调用、工作区隔离、命令白名单和测试闭环。

不要写没有验证过的数字，不要把 Mock 模式写成生产系统，不要把团队实习代码写成个人独立完成。

八、下一步升级路线

1. 接入真实 Embedding 和 Rerank
2. 增加 FastAPI 接口和 Vue 页面
3. 增加模型结构化输出和 Tool Calling
4. 增加权限、日志、超时和重试
5. 建立 20 条离线测试问题
6. 记录每次修改后的效果和失败案例

7. 将网站部署到阿里云 OSS 或 ECS